

PrepME AI — Day 8 Notes (useEffect, Auto Login, Logout & Frontend Delete Notes)

1. Session Goal

Today's goal was to make the Notes application behave more like a real product instead of a learning demo.

Before today:

```
Login
↓
Manually Click Get Profile
↓
Manually Click Get Notes
```

After today:

```
Login
↓
Refresh Page
↓
Profile Loads Automatically
↓
Notes Load Automatically
```

We also implemented:

```
Logout
Delete Notes From UI
```

2. Understanding useEffect()

New React Hook:

```
useEffect()
```

Purpose:

Run code automatically
when something happens

Think of it as:

Automatic Function Execution

Basic Example

```
useEffect(() => {  
  console.log("Hello")  
}, [])
```

Output:

Hello

What Happens?

Component loads:

```
React Component Starts  
↓  
useEffect Runs  
↓  
Hello Printed
```

3. Understanding Dependency Array

Example:

```
useEffect(() => {  
  
}, [])
```

The empty array:

```
[]
```

means:

Run only once
when component loads

Important Questions

Page Loads?

Runs 1 time

☒ Yes

Input Changes?

Does NOT run again

because:

Dependency array is empty

State Changes?

Does NOT run again

because:

Component re-renders
but useEffect has []

Comparison

Runs Once

```
useEffect(() => {  
  
}, [])
```

Meaning:

Component Mount

Runs Every Render

```
useEffect(() => {  
  
})
```

Meaning:

Every Re-render

Runs When Variable Changes

```
useEffect(() => {  
  
}, [notes])
```

Meaning:

Whenever notes changes

4. Auto Login Persistence

Before:

Refresh Page
↓
Everything Disappears

User had to click:

Get Profile
Get Notes

again.

Solution

Implemented:

```
useEffect(() => {  
  
  const token =  
    localStorage.getItem("token")  
  
  if (token) {  
  
    getProfile()  
  
    getNotes()  
  
  }  
  
}, [])
```

Flow

```
Page Opens
↓
Check localStorage
↓
Token Exists?
↓
YES
↓
Get Profile
↓
Get Notes
↓
Display Data
```

Real World Example

Applications using same concept:

```
Gmail
LinkedIn
Amazon
Netflix
Instagram
```

You remain logged in after refresh because:

```
Token already exists
```

5. localStorage Revision

Store:

```
localStorage.setItem(
  "token",
```

```
    token  
  )
```

Purpose:

Save JWT

Read

```
localStorage.getItem(  
  "token"  
)
```

Purpose:

Retrieve JWT

Remove

```
localStorage.removeItem(  
  "token"  
)
```

Purpose:

Delete JWT

Used for logout.

6. Logout Feature

Implemented:

```
async function handleLogout() {  
  
  localStorage.removeItem("token")  
  
  setProfile(null)  
  
  setNotes([])  
  
  setMessage(  
    "Logged out successfully"  
  )  
  
}
```

Why Remove Token?

Current authentication depends on:

JWT Token

Without token:

Backend cannot identify user

Logout Flow

```
Click Logout  
↓  
Remove JWT  
↓  
Clear Profile  
↓  
Clear Notes  
↓  
User Logged Out
```

7. Why Clear Profile?

Before logout:

```
Email:  
test@gmail.com
```

If profile remains:

```
Old user data still visible
```

Bad user experience.

Solution

```
setProfile(null)
```

Result:

```
Profile disappears
```

8. Why Clear Notes?

Before logout:

```
JWT Notes  
React Notes  
MongoDB Notes
```

If notes remain:

```
Logged out user  
still sees data
```

Bad UX.

Solution

```
setNotes([])
```

Result:

```
Notes disappear
```

9. JWT Lifecycle (Complete)

Today we completed understanding the entire JWT flow.

```
Login
↓
Generate Token
↓
Store Token
↓
Send Token
↓
Verify Token
↓
Access Protected Routes
↓
Logout
↓
Remove Token
↓
Protected Routes Fail
```

This is how authentication works in most modern applications.

10. What Happens After Logout?

Example:

```
localStorage.removeItem(  
  "token"  
)
```

Then:

```
getNotes()
```

tries:

```
const token =  
localStorage.getItem("token")
```

Result:

```
null
```

Request:

```
Authorization: null
```

Backend:

```
req.header(  
  "Authorization"  
)
```

returns:

```
null
```

Middleware:

No Token Found

Response:

```
{
  "message":
    "No token provided"
}
```

11. Frontend Delete Notes

Before:

Delete only worked in Postman

After:

Delete works directly from UI

New Function

Created:

```
async function deleteNote(id)
```

Purpose:

Delete selected note

Delete Request

```
axios.delete(  
  `/notes/${id}`  
)
```

Example

Suppose:

```
{  
  _id: "123",  
  title: "Learn JWT"  
}
```

Call:

```
deleteNote("123")
```

Backend receives:

```
DELETE /notes/123
```

Flow

```
Click Delete  
↓  
Send Note ID  
↓  
Delete API  
↓  
MongoDB  
↓  
Remove Note
```

↓

Refresh Notes

12. React Dynamic Event Handling

New React pattern:

```
onClick={() =>
  deleteNote(note._id)
}
```

Purpose:

```
Pass current note ID
when button is clicked
```

Why Not This?

Wrong:

```
onClick={
  deleteNote(note._id)
}
```

Problem:

```
Function executes immediately
during render
```

Not when clicked.

Correct

```
onClick={() =>
  deleteNote(note._id)
}
```

Meaning:

```
Wait
↓
User Clicks
↓
Run Function
```

13. Understanding notes.map()

Current code:

```
notes.map((note) => (
  ...
))
```

Purpose:

```
Convert array
into UI
```

Example

Data:

```
[
  {
    title: "JWT"
```

```
    },  
  
    {  
      title: "React"  
    }  
  ]  
}
```

Output:

```
JWT  
  
React
```

14. Dynamic Buttons

Each note:

```
{  
  _id,  
  title  
}
```

gets its own:

```
Delete Button
```

Example:

```
JWT Notes      [Delete]  
  
React Notes    [Delete]
```

Each button receives a different:

```
note._id
```

Example

Notes:

```
[
  {
    _id: "111",
    title: "JWT"
  },
  {
    _id: "222",
    title: "React"
  }
]
```

Click:

Delete React

React executes:

```
deleteNote("222")
```

15. Current Project Status

Backend:

Signup	✓
Login	✓
JWT	✓
Middleware	✓
Protected Routes	✓
Authorization	✓
Create Note	✓
Get Notes	✓
Update Note	✓
Delete Note	✓

Frontend:

Signup	✓
Login	✓
Get Profile	✓
Create Note	✓
Get Notes	✓
Display Notes	✓
Auto Login	✓
Logout	✓
Delete Note	✓

16. Skills Learned Today

React:

- ✓ `useEffect()`
- ✓ Dependency Arrays
- ✓ Auto API Calls
- ✓ Event Handlers
- ✓ Dynamic Button Actions

Authentication:

- ✓ Persistent Login
- ✓ Logout Flow
- ✓ Token Lifecycle

Frontend Integration:

- ✓ Delete Notes
- ✓ Auto Fetch Data

17. Most Important Learning Today

A React application should not rely on users clicking buttons for everything.

Good applications:

```
Open App
↓
Load Required Data
↓
Display UI Automatically
```

This is achieved using:

```
useEffect()
```

18. Next Session Plan

Frontend CRUD Completion:

```
Edit Note Button
Update Note From UI
```

Additional Improvements:

```
Hide buttons when logged out
Show login-required messages
Better UI structure
```

Key Takeaway

```
useEffect()
=
```

Automatic Function Execution

localStorage

=

Persistent Browser Storage

Logout

=

Remove JWT + Clear UI

Delete Note

=

Pass Note ID

→ Call API

→ Refresh UI

Current Project Progress:

~80% Complete

Remaining:

~20%

The remaining work focuses on making the application production-like and interview-ready.